

基於 ARM NEON 的高效能平行化 Sine/Cosine 演算法優化

從數學原理到 SIMD 硬體加速實作

1. 為什麼需要加速？(Motivation)

- **性能瓶頸**：在嵌入式系統中，標準庫 `<math.h>` 的 `sinf()` / `cosf()` 是純純的 Scalar 運算。
- **大量運算需求**：訊號處理 (DSP)、機器人運動學 (Inverse Kinematics) 往往需要對整個數組進行三角運算。
- **SIMD 的優勢**：ARM NEON 提供 128-bit 暫存器，單次指令可處理 4 個 `float`，理論吞吐量提升 4 倍。

2. 核心演算法：Cephes 數學庫

本專案基於經典的 **Cephes** 演算法進行向量化改造：

- **逼近策略**：不使用展開項極多的泰勒級數，而是使用 **Minimax 近似多項式**。
- **值域縮減 (Range Reduction)**：將輸入 x 映射至 $[0, \pi/4]$ 區間。
- **對稱性利用**：透過整數運算與位元遮罩判定象限，決定最終結果的符號。

3. 精度關鍵：Cody–Waite 值域縮減

當 x 很大時，直接用 `x % pi` 會導致精確度喪失。我們採用 **Cody–Waite** 策略：

- **多段 π 常數**：將 $\pi/4$ 分解為高精度、低誤差的三個部分 (DP1, DP2, DP3)。
- **公式**：

$$x_{reduced} = ((x - y \cdot DP1) - y \cdot DP2) - y \cdot DP3$$

- **效益**：確保在 $x < 8192$ 時仍能維持單精度浮點數的極限精度。

4. ARM NEON 加速架構

我們實作了向量化的 `sincos_ps(float32x4_t x, ...)` :

```
// 核心平行化循環
for (i = 0; i < vec_count; i += 4) {
    // 1. 同時載入 4 個輸入
    float32x4_t v_in = vld1q_f32(&input[i]);
    // 2. SIMD 平行計算 sin 與 cos
    sincos_ps(v_in, &v_sin, &v_cos);
    // 3. 同時存入 4 個結果
    vst1q_f32(&sin_out[i], v_sin);
    vst1q_f32(&cos_out[i], v_cos);
}
```

5. NEON 優化技巧 Deep Dive

為了極致效能，我們避開了條件分支，改用硬體原語：

- **FMA (Fused Multiply-Add):**

使用 `vfmaq_f32` 減少指令週期，同時提高多項式累加的精確度。

- **無分支邏輯 (Branch-less):**

利用 `vbslq_f32` (Bitwise Select) 根據象限遮罩選擇結果，避免 CPU 分支預測錯誤 (Branch Misprediction)。

- **位元運算:**

使用 `veorq_u32` 快速切換符號位，取代傳統的 `if (x < 0) ...`。

6. 精度分析：實測數據

將 NEON 版與標準 C 庫 (`sinf` / `cosf`) 進行對比 ($N = 66,820$) :

指標	數值
最大絕對誤差 (Max Error)	7.52×10^{-08}
均方根誤差 (RMSE)	2.25×10^{-08}
測試結果	PASSED (優於 10^{-5} 目標)

結論：在肉眼不可見的精度損失下，換取極大的效能提升。

7. 效能表現：3.05x 加速

在 ARM64 環境下的基準測試結果：

- 資料規模：66,820 個元素
- Scalar Throughput：554.16 M/s
- NEON Throughput：1689.57 M/s
- 實測加速比：3.05 倍

備註：受限於記憶體對齊與 Cody–Waite 複雜指令，雖未達 4x 理論值，但已大幅超越傳統寫法。

8. 開發者實作建議

1. 記憶體對齊 (Alignment) :

確保 `float*` 對齊 16-byte，否則 `vld1q` 可能觸發效能懲罰。

2. 尾端處理 (Tail Handling) :

處理 `count % 4 != 0` 的剩餘數據，使用傳統 `sinf` 補足。

3. 架構分歧：

AArch64 支援 `vrndmq_f32` (Floor)，舊型 AArch32 需手動模擬。

9. 總結 (Summary)

- **高效能**：透過 NEON SIMD 實現 300% 的吞吐量增長。
- **高精確度**：採用 Cody–Waite 值域縮減與 Cephes 多項式。
- **無分支設計**：最大化處理器流水線 (Pipeline) 效率。
- **適用場景**：嵌入式視覺、即時感測器融合、高效能物理模擬。